

DIGITAL TWIN, MQTT COMMUNICATION & XARM5 ROBOT CONTROL

BALOG BOGDAN GHEORGHE

WHAT IS A DIGITAL TWIN?

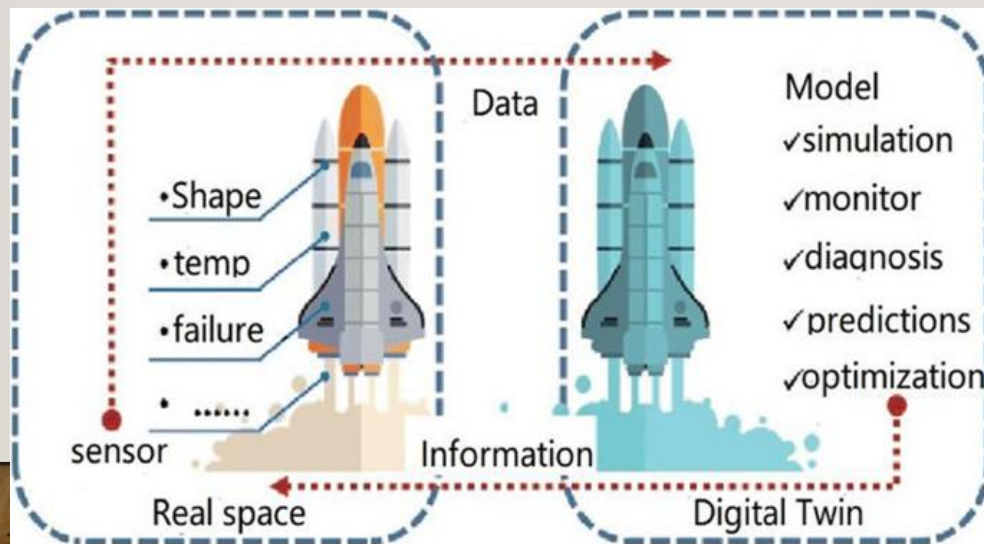
- A **Digital Twin** is a **virtual, real-time digital representation of a physical object, process, system, or environment**, continuously updated with data collected from sensors, devices, or other information sources.

It enables monitoring, simulation, analysis, and optimization of the lifecycle.



HISTORY & EVOLUTION

- 2002: introduced Digital Twin as concept
- Industry 4.0 expanded to factories and robotics
- Now used in healthcare, energy, cities



CORE COMPONENTS

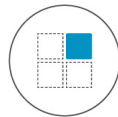
- Physical system (robot, machine, vehicle)
- Digital model (simulation, CAD, 3D)
- Data connection (sensors, protocols, IoT)
- Analytics & AI

TYPES OF DIGITAL TWINS

- Component Twin → single part (e.g., gripper)
- System Twin → full robot or machine
- Process Twin → entire factory workflow

Digital Twins: The 4 types

Example: Car factory



Component/Parts Twins

E.g. rotor, bulb



Asset Twins

E.g. engine or pump



System/Unit Twins

Combines all production units



Process Twins

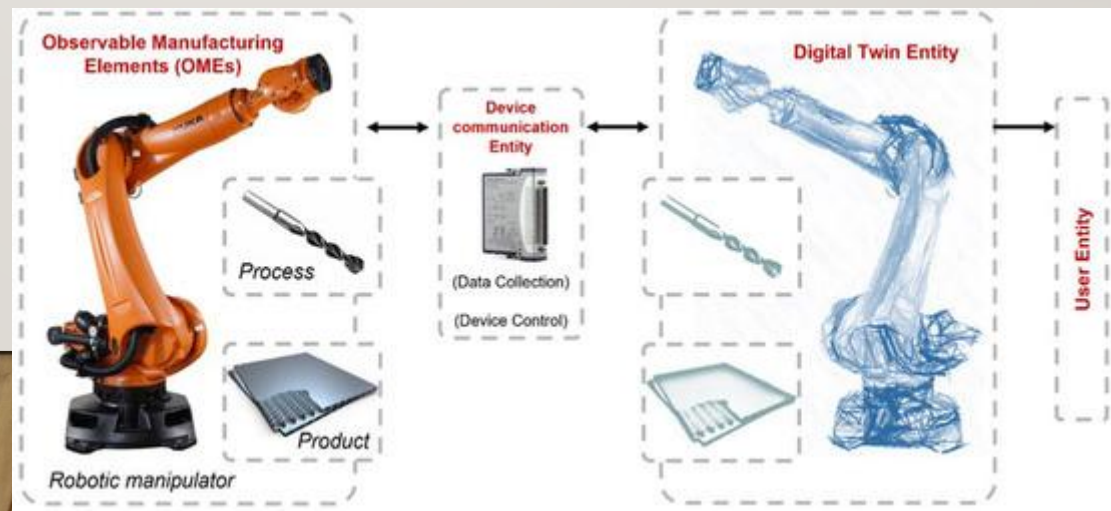
E.g. entire manufacturing process

BENEFITS OF DIGITAL TWIN

- Predictive maintenance
- Real-time monitoring
- Safe testing of scenarios
- Performance optimization

APPLICATIONS IN ROBOTICS

- Testing robot control algorithms virtually
- Training AI in simulation before deployment
- Monitoring wear of joints
- Creating dashboards

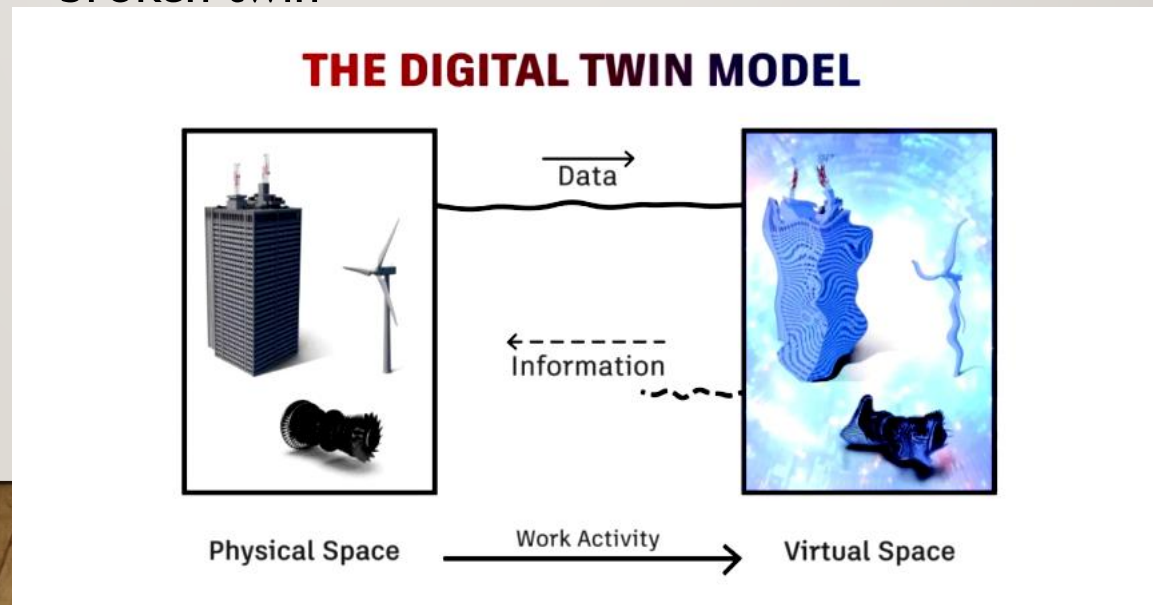


CHALLENGES

- Requires real-time data
- High bandwidth needed for complex systems
- Security of communication
- Accurate modeling of physics

WHY DATA MATTERS

- Twin accuracy depends on data quality
- Needs: reliability, low latency, accuracy
- Bad data = broken twin



ROLE OF COMMUNICATION

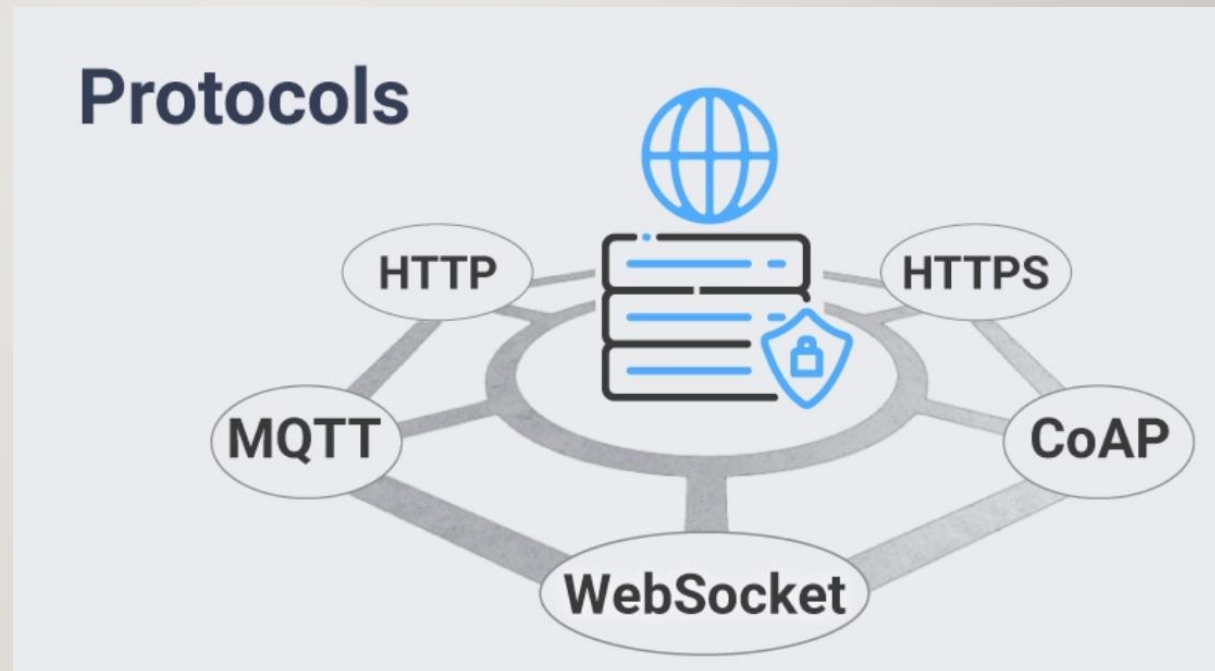
- Connects physical world $\leftrightarrow$ digital twin
- Transmission = lifeline of the twin
- Without data, twin is static

TRANSMISSION REQUIREMENTS

- Accuracy (no corruption)
- Reliability (no loss)
- Scalability (support many devices)
- Low latency (real-time)

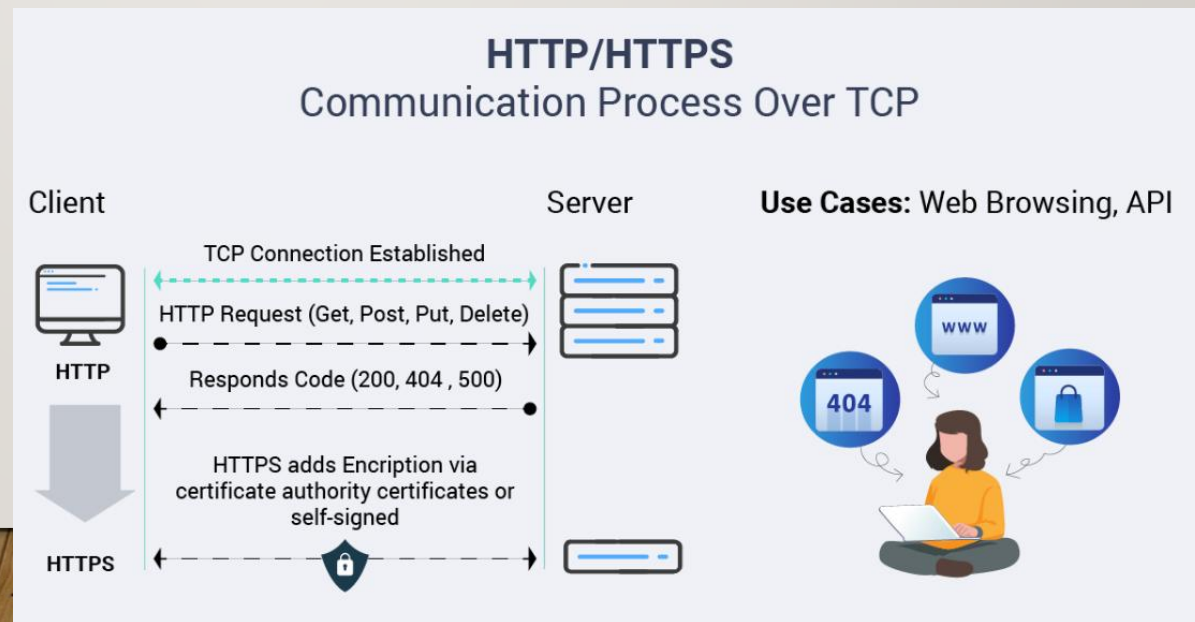
PROTOCOL LANDSCAPE

- HTTP/REST
- WebSockets
- CoAP
- MQTT (focus of lecture)



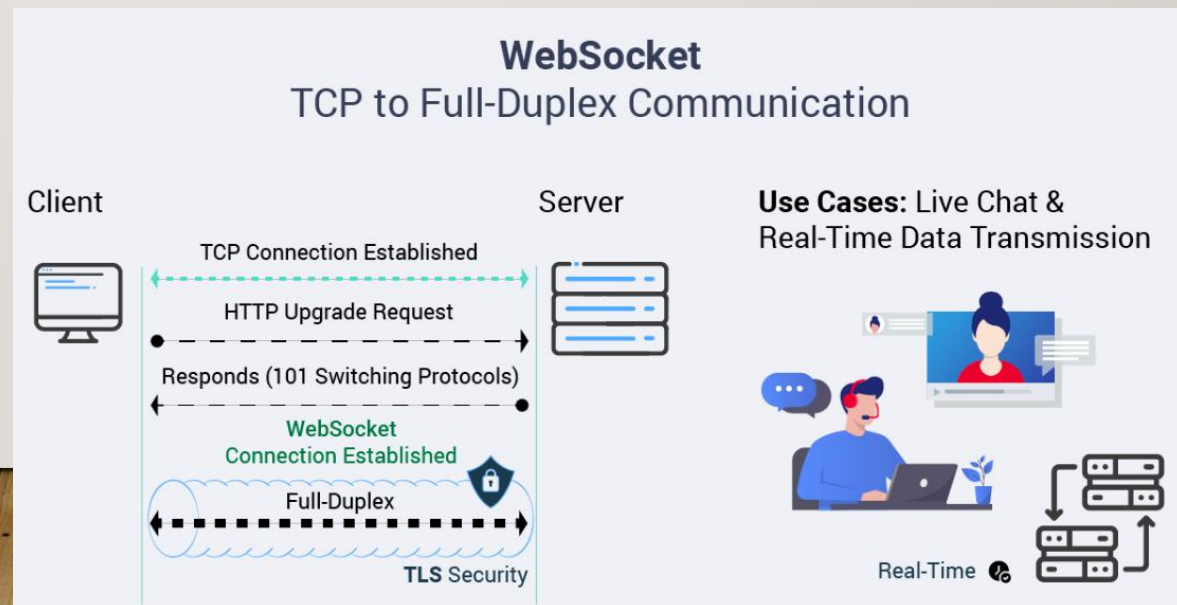
HTTP/REST

- Request/Response model
- Good for config, not real-time
- High overhead



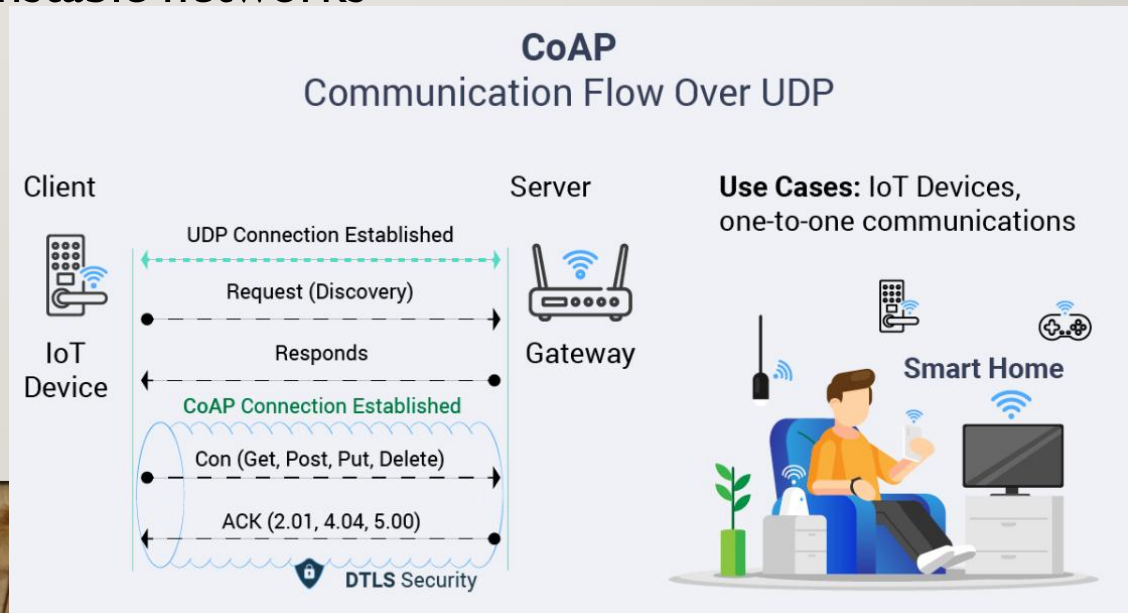
WEBSOCKETS

- Bi-directional, real-time
- Used in dashboards
- More complex than MQTT



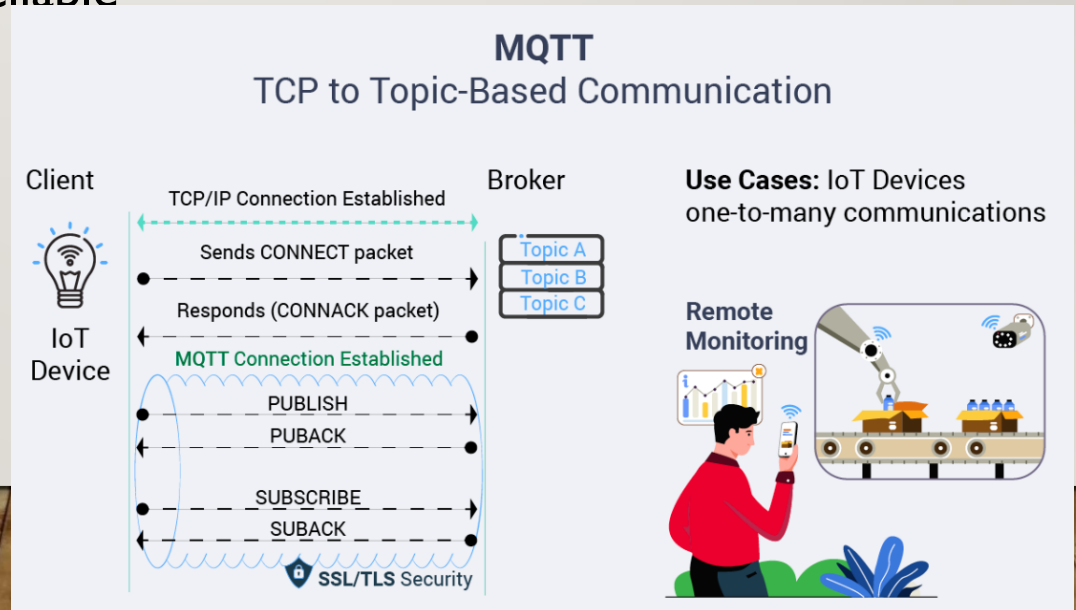
COAP

- Lightweight, REST over UDP
- For constrained devices
- Less reliable on unstable networks



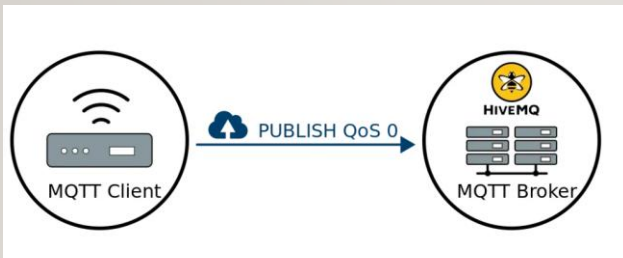
MQTT BASICS

- Publish/Subscribe architecture
- Broker in the middle
- Lightweight, fast, reliable
- Designed for IoT



MQTT ADVANTAGES

- Minimal bandwidth use
- Works with unstable connections
- QoS options: At most once, At least once, Exactly once



Quality of Service level 0: delivery at most once



Quality of Service level 1: delivery at least once



MQTT Quality of Service level 2: delivery exactly once

MQTT IN ROBOTICS

- Robots publish sensor data
- Twins subscribe & update models
- Twins send commands back
- Flexible, scalable

EXAMPLE FLOW

- Robot → publishes sensors
- Twin → updates 3D simulation
- Twin → publishes commands
- Robot executes

WHY MQTT

- Easy to learn
- Works on PC, robot, and mobile
- Visual tools: MyMQTT, MQTT Explorer

PUBLIC BROKER FOR TESTS

- broker.emqx.io
- Port 1883
- Free to use for demos/ !!! Check before use in real application

PROGRAM GOAL

- Control xArm5 gripper via MQTT
- One topic: xarm5/cmd/gripper
- Payload "1" = Close, "2" = Open

SHEBANG & IMPORTS

- `#!/usr/bin/env python3`
- `socket, struct` → networking
- `threading` → background loop
- `XArmAPI` → robot SDK

MINIMAL MQTT CLIENT

- Pure Python implementation
- Handles connect, subscribe, publish
- No external libraries required

CODE PYTHON – ROBOT XARM5

MINIMAL MQTT CLIENT (QoS0)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import time, threading, socket, struct
from xarm.wrapper import XArmAPI

# ----- Minimal MQTT Client (QoS0) -----
class MiniMQTTClient:
    class _Msg:
        def __init__(self, topic, payload):
            self.topic = topic
            self.payload = payload

    def __init__(self, client_id=None, keepalive=30):
        self.client_id = client_id or f"mini-{int(time.time())}"
        self.keepalive = keepalive
        self.sock = None
        self.on_connect = None
        self.on_message = None
        self.on_disconnect = None
        self._stop = False
        self._last_io = time.time()
        self._lock = threading.Lock()

    def connect(self, host, port=1883, keepalive=30):
        self.keepalive = keepalive
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.settimeout(5.0)
        self.sock.connect((host, port))
        self.sock.settimeout(0.2)
        self._send_connect()
        # wait for CONNACK
        end = time.time() + 5
        while time.time() < end:
            pkt = self._recv_packet()
            if pkt and pkt[0] == 2: # CONNACK
                if self.on_connect:
                    # rc is second byte of CONNACK payload
                    self.on_connect(self, None, {}, pkt[2][1])
                return pkt[2][1]
        return 1
```

```
def subscribe(self, topic, qos=0):
    pid = struct.pack("!H", 1)
    payload = pid + self._encode_str(topic) + b"\x00"
    self._send(0x82, payload)

def loop_forever(self):
    while not self._stop:
        # light keepalive: send PINGREQ if quiet
        if self.keepalive and (time.time() - self._last_io) > self.keepalive * 0.8:
            try:
                with self._lock:
                    self.sock.sendall(b"\xC0\x00") # PINGREQ
                    self._last_io = time.time()
            except:
                break
        pkt = self._recv_packet()
        if pkt is None:
            continue
        ptype, flags, payload = pkt
        if ptype == 3: # PUBLISH
            topic, msg_payload = self._parse_publish(payload)
            if self.on_message:
                self.on_message(self, None, MiniMQTTClient._Msg(topic, msg_payload))
```

CODE PYTHON – ROBOT XARM5

MINIMAL MQTT CLIENT (QOS0)

```
# --- internals ---
def _encode_str(self, s):
    b = s.encode()
    return struct.pack("!H", len(b)) + b

def _encode_varint(self, n):
    out = bytearray()
    while True:
        d = n % 128
        n //= 128
        if n > 0: d |= 0x80
        out.append(d)
        if n == 0: break
    return bytes(out)

def _send(self, header, payload):
    r1 = self._encode_varint(len(payload))
    with self._lock:
        self.sock.sendall(bytes([header]) + r1 + payload)
    self._last_io = time.time()

def _send_connect(self):
    payload = (
        self._encode_str("MQTT") + b"\x04" + b"\x02" +
        struct.pack("!H", self.keepalive) + self._encode_str(self.client_id)
    )
    self._send(0x10, payload)

def _recv_exact(self, n):
    buf = bytearray()
    try:
        while len(buf) < n:
            c = self.sock.recv(n - len(buf))
            if not c: return None
            buf.extend(c)
        return bytes(buf)
    except:
        return None
```

```
def _recv_packet(self):
    try: b1 = self.sock.recv(1)
    except: return None
    if not b1: return None
    mult, val = 1, 0
    while True:
        try: b = self.sock.recv(1)
        except: return None
        if not b: return None
        enc = b[0]
        val += (enc & 127) * mult
        if (enc & 128) == 0: break
        mult *= 128
    payload = self._recv_exact(val)
    return (b1[0] >> 4, b1[0] & 0x0F, payload)

def _parse_publish(self, payload):
    if not payload or len(payload) < 2: return "", ""
    tlen = struct.unpack("!H", payload[0:2])[0]
    topic = payload[2:2+tlen].decode(errors="replace")
    idx = 2 + tlen
    msg = payload[idx:]
    return topic, msg.decode(errors="replace")
```

CODE PYTHON – ROBOT XARM5

ROBOT + MQTT BRIDGE

```
# ----- Robot + MQTT Bridge -----
BROKER = "broker.emqx.io"
PORT = 1883
TOPIC_GRIPPER = "xarm5/cmd/gripper" # one topic only

def on_connect(c, u, f, rc):
    print("MQTT connected, rc =", rc)
    c.subscribe(TOPIC_GRIPPER)
    print("Subscribed to", TOPIC_GRIPPER)

def on_message(c, u, m):
    payload = m.payload.strip()
    print(f"MQTT {m.topic} {payload}")
    # "1" -> CLOSE, "2" -> OPEN
    if payload == "1":
        # CLOSE
        arm.set_gripper_position(70, wait=True, speed=1000, auto_enable=True)
        print("Gripper CLOSED")
    elif payload == "2":
        # OPEN
        arm.set_gripper_position(600, wait=True, speed=1000, auto_enable=True)
        print("Gripper OPENED")
    else:
        print("Ignored payload (expect '1' or '2')")
```

```
if __name__ == "__main__":
    # Connect to robot
    arm = XArmAPI("192.168.1.199", baud_checkset=False)
    arm.clean_warn(); arm.clean_error()
    arm.motion_enable(True); arm.set_mode(0); arm.set_state(0)
    time.sleep(1)

    # Connect to MQTT
    client = MiniMQTTClient()
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, PORT, 30)

    threading.Thread(target=client.loop_forever, daemon=True).start()

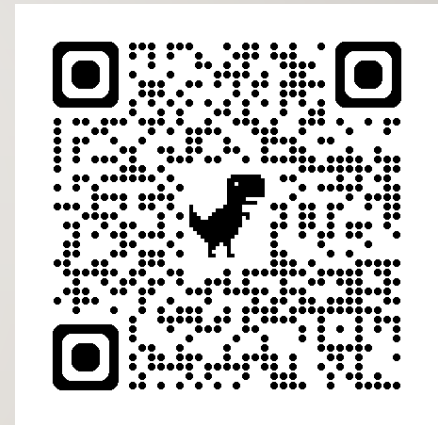
    print("Ready. Send MQTT messages to:")
    print("-", TOPIC_GRIPPER, 'with payload "1" ☐ CLOSE, "2" ☐ OPEN')

    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("Stopped.")
```

SUBSCRIBING TO TOPIC

- `c.subscribe("xarm5/cmd/gripper")`
- Robot listens on one topic
- Messages arrive from PC or mobile

<https://play.google.com/store/apps/details?id=at.tripwire.mqtt.client>



MyMQTT

instant:solutions OG

Conține anunțuri · Achiziții în aplic.

100 K+
Descărcări

 PEGI 3

Instalează pe mai multe dispozitive

 Trimite



ON_MESSAGE FUNCTION

- if payload == '1': close gripper
- if payload == '2': open gripper



Publish

Topic

Message

Advanced Save Publish

xarm5/cmd/gripper

2

Dashboard Subscribe Publish

MAIN LOOP

- Connects to robot IP
- Connects to broker
- Listens forever for messages

CLASSROOM DEMO FLOW

- Run script on robot
- Students publish from app
- Robot gripper moves in real time

KEY TAKEAWAYS

- Digital Twin depends on real-time, reliable data
- MQTT is ideal for IoT and robotics
- Python script bridges MQTT with robot SDK
- Mobile apps enable easy demos

NEXT STEPS

1. Secure MQTT with auth & TLS
2. Add more robot commands(joints data, temperature,TCP position ...)
3. Build full digital twin simulation
4. Enable bi-directional control
5. Optional application can add simulation control via Robodk, other simulation and modeling application, pure Python application/visualizer or XR application integration.